

CSCE 711 APPLIED CRYPTOGRAPHY · MODULE 04 · LECTURE NOTES

Authenticated Encryption

The only acceptable default.

MODULE OUTCOMES

- Explain what a MAC provides and how to use one.
- Explain the composition orders and which is safe.
- Explain AEAD and associated data.
- Name the AEAD constructions and choose between them.
- Explain why verification must precede use.

1 The MAC

A MAC IS A KEYED TAG THAT ONLY A KEYHOLDER CAN PRODUCE OR VERIFY

Given a key and a message it produces a short tag — typically 16 or 32 bytes — and **without the key, producing a valid tag for any message the keyholder has not already tagged is computationally infeasible.**

So it provides integrity and authentication together (Module 01 §1): **a valid tag means the message was not modified and that it came from somebody holding the key** — two of the four properties, from one primitive.

HMAC is the construction to know. It is built from any hash function, is extremely well understood, and is specifically not vulnerable to the length-extension problem that a naive hash-the-key-and-message construction has (Module 05 §3) — which is exactly why HMAC's nested structure looks odd.

And a hash is not a MAC. An unkeyed hash can be recomputed by anyone, including the adversary who modified the message, so it detects accidental corruption and nothing adversarial whatsoever — which is the single most common misuse of a hash function and appears in production systems as 'we hash it to make sure it wasn't tampered with'.

TAG COMPARISON MUST BE CONSTANT-TIME

A normal string or byte comparison returns as soon as it finds a difference — that is what makes it fast — so the time it takes reveals how many leading bytes matched.

Which lets an adversary forge a tag one byte at a time. Guess the first byte in all 256 ways, keep whichever request took measurably longest, then move to the second byte — 256×16 attempts instead of 2^{128} , which is the difference between infeasible and an afternoon.

So every serious library provides a constant-time comparison — `hmac.compare_digest` in Python, `sodium_memcmp`, Go's `subtle.ConstantTimeCompare` — and using `==` or `memcmp` on an authentication tag is a defect, however obviously correct it looks in review.

This is Module 11 §1's whole subject appearing early, because it is the one timing attack that shows up in ordinary application code rather than inside a primitive implementation — so it is the one you are personally likely to write.

2 Composition

Order	What it does	Verdict
Encrypt-then-MAC	Encrypt the plaintext, then MAC the ciphertext (and the IV, and the header).	Sound. An invalid ciphertext is rejected before decryption is attempted, which is the practical property that matters.
MAC-then-encrypt	MAC the plaintext, then encrypt the plaintext and the tag together.	You must decrypt in order to verify, so you process attacker-controlled data before authenticating it — hazardous, and the source of several real attacks.
Encrypt-and-MAC	MAC the plaintext, encrypt the plaintext, send both.	Unsound generically — the tag is a deterministic function of the plaintext, so identical plaintexts produce identical tags, leaking equality.

AND THE CORRECT ANSWER IS TO COMPOSE NOTHING

Encrypt-then-MAC is sound and still has to be built correctly: two independent keys, properly derived rather than reused; the right byte ranges covered; the tag checked before any parsing takes place.

And every one of those is a place to make a mistake that no functional test will catch — a MAC computed over the ciphertext but not over the IV, for instance, which permits an adversary to change the IV and thereby the first plaintext block, and which has appeared in deployed systems more than once.

So use an AEAD primitive, which performs the composition internally and exposes a single call.

There is no order to get wrong, because there is no order to choose — the decision was made once, by people who prove things, and baked into the construction.

Which is why §2 is diagnostic rather than instructional. You learn the three orders so that you can audit an existing system and recognise which one it implements, not so that you can implement one yourself — and the same framing applies to Module 03 §4.

3 AEAD

```
encrypt(key, nonce, plaintext, associated_data)
  -> ciphertext_with_tag

decrypt(key, nonce, ciphertext_with_tag, associated_data)
  -> plaintext, OR an error. Never both.
```

ASSOCIATED DATA is authenticated but NOT encrypted. It is for context that must not be tampered with and must stay readable:

- a record identifier, so a valid ciphertext cannot be moved to a different record
- a version or algorithm label, so it cannot be downgraded
- a recipient identifier, so a message cannot be replayed at someone else
- routing headers that intermediaries must read

THE DECRYPTION CONTRACT: on tag failure you receive an error and NO plaintext. Not partial output, not a "probably fine". This is the property the whole construction exists to provide.

Associated data binds the ciphertext to its context, and it is substantially underused. The record-identifier case is the one that lands: without it, an adversary with write access to your database can move a validly encrypted, validly authenticated value from row 7 to row 9 — every cryptographic check passes, and the data is now wrong in a way your application will trust. **With the row identifier as associated data, that ciphertext fails to authenticate in its new position.**

- **AES-GCM. The default wherever AES hardware acceleration exists** — fast, standardised, and implemented everywhere. **Nonce reuse is catastrophic and additionally forfeits the authentication key itself**, which is worse than for a plain stream cipher (Module 11 §2).
- **ChaCha20-Poly1305. The default where AES hardware is absent** — fast in software and straightforward to implement in constant time (Module 03 §3). Specified in TLS 1.3 for this reason.
- **AES-GCM-SIV. Nonce-misuse resistant:** a repeated nonce leaks only whether two plaintexts were identical, rather than destroying confidentiality and authenticity outright. **Use it wherever you cannot genuinely guarantee nonce uniqueness.**
- **XChaCha20-Poly1305. A 192-bit nonce, which makes a randomly generated nonce safe indefinitely** by the birthday bound — **removing the counter management problem entirely**, which is the right trade for most distributed systems.
- **And libsodium's crypto_secretbox**, which makes every one of these decisions on your behalf. **The misuse-resistant options are underused:** if your nonce discipline depends on distributed coordination (CSCE 678 Module 05), XChaCha20 or GCM-SIV removes that dependency, which is worth more than the marginal performance.

4 The discipline

VERIFY THE TAG BEFORE YOU PARSE, DECOMPRESS, OR LOG ANYTHING

The guarantee is only worth something if you act on it. Code that decrypts, parses the result, and *then* checks the tag has already processed attacker-controlled input — through a parser, which is exactly where the memory-safety and injection bugs are (CSCE 713).

And the leak need not be the plaintext itself. A distinguishable error message, a measurable timing difference, a log line, or any observable difference between failure modes is enough to construct a decryption oracle — which is precisely how padding oracle attacks recover plaintext without the key, using nothing but the server's error behaviour.

So: one error, indistinguishable, for every failure. Not 'bad padding' versus 'bad tag' versus 'malformed length' versus 'unknown key identifier' — the distinction is the vulnerability, and the helpful error message is the defect. Log the detail internally; return one opaque failure.

A proper AEAD library enforces this by returning nothing at all on failure — no partial plaintext, no diagnostic — which is another case of the API preventing the mistake rather than the documentation warning about it, and is the pattern Module 01 §4 asks you to look for when choosing between libraries.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Boneh & Shoup, chapters 6 through 9 (free)	MACs and authenticated encryption with the proofs — including why encrypt-then-MAC is generically secure and the others are not.
Aumasson — Serious Cryptography, chapters 7 through 9	§1 through §3 at engineering level , with the construction comparisons laid out practically.
RFC 5116 and RFC 8452 (AEAD interface; AES-GCM-SIV) (free)	§3's interface as standardised , and the misuse-resistant construction's specification and rationale.
Rizzo & Duong on padding oracles (free)	§4's argument, demonstrated — plaintext recovery from nothing but distinguishable error behaviour.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write a MAC verification with ==** and measure the timing difference across first-byte guesses.

- 2 **Replace it with the constant-time comparison** and measure again.
- 3 **Find one place in code you have access to** where a hash is used where a MAC is needed.
- 4 **Implement encrypt-then-MAC manually**, then list every detail you had to get right.
- 5 **Deliberately omit the IV from the MAC** and show what an adversary can then change.
- 6 **Replace the whole thing with one AEAD call** and compare the line counts.
- 7 **Use associated data to bind a ciphertext to a record identifier**, then try moving it.
- 8 **Confirm your AEAD library returns nothing on tag failure**, rather than partial plaintext.
- 9 **Audit your error handling** for distinguishable decryption failures.
- 10 **Pick between XChaCha20 and AES-GCM for your own system** and justify it on nonce management.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does a MAC provide, and why is a hash not one?
- 2 Why does HMAC have its particular nested structure?
- 3 Why must tag comparison be constant-time, and what is the attack's cost?
- 4 Give the three composition orders and the verdict on each.
- 5 Why is encrypt-then-MAC sound, in practical terms?
- 6 Why should you compose nothing at all?
- 7 What is associated data for, and give the record-identifier example.
- 8 State the AEAD decryption contract.
- 9 Name four AEAD constructions and when to use each.
- 10 Why must every decryption failure look identical?